OpenCV Module — Part 2
Practicals 10 – 15

Module	OpenCV – Computer Vision with Python (Part 2)
Practicals	10 through 15
Library	OpenCV 4.13.0 (cv2)
Language	Python 3.12
Date	March 20, 2026
Timestamp	2026-03-20 06:13:58

#	Practical	Key Functions / Concepts
10	Morphological Operations	erode, dilate, MORPH_OPEN, MORPH_CLOSE, MORPH_GRADIENT, TOPHAT, BLA
11	Image Arithmetic & Bitwise Ops	add, subtract, convertScaleAbs, addWeighted, bitwise_and/or/xor/not
12	Template Matching	matchTemplate (6 methods), minMaxLoc, multi-template, JET heatmap
13	Color-based Segmentation	inRange, HSV masking, GrabCut foreground extraction
14	Feature Detection & Description	Harris, Shi-Tomasi, FAST, ORB, BFMatcher, drawMatches
15	Image Pyramids & Optical Flow	pyrDown/Up, Laplacian pyramid, LK sparse flow, Farneback dense

Practical 10: Morphological Operations

Executed: 2026-03-20 06:13:58 | OpenCV 4.13.0 | Python 3.12 | Ubuntu 24.04

Morphological operations process images based on shapes. Applied to binary images, they use a **structuring element** (kernel) to probe and transform the image. **Erosion** shrinks bright regions; **Dilation** expands them. **Opening** (erode→dilate) removes noise; **Closing** (dilate→erode) fills gaps. **Gradient** extracts edges; **Top Hat** and **Black Hat** enhance fine details.

Code:

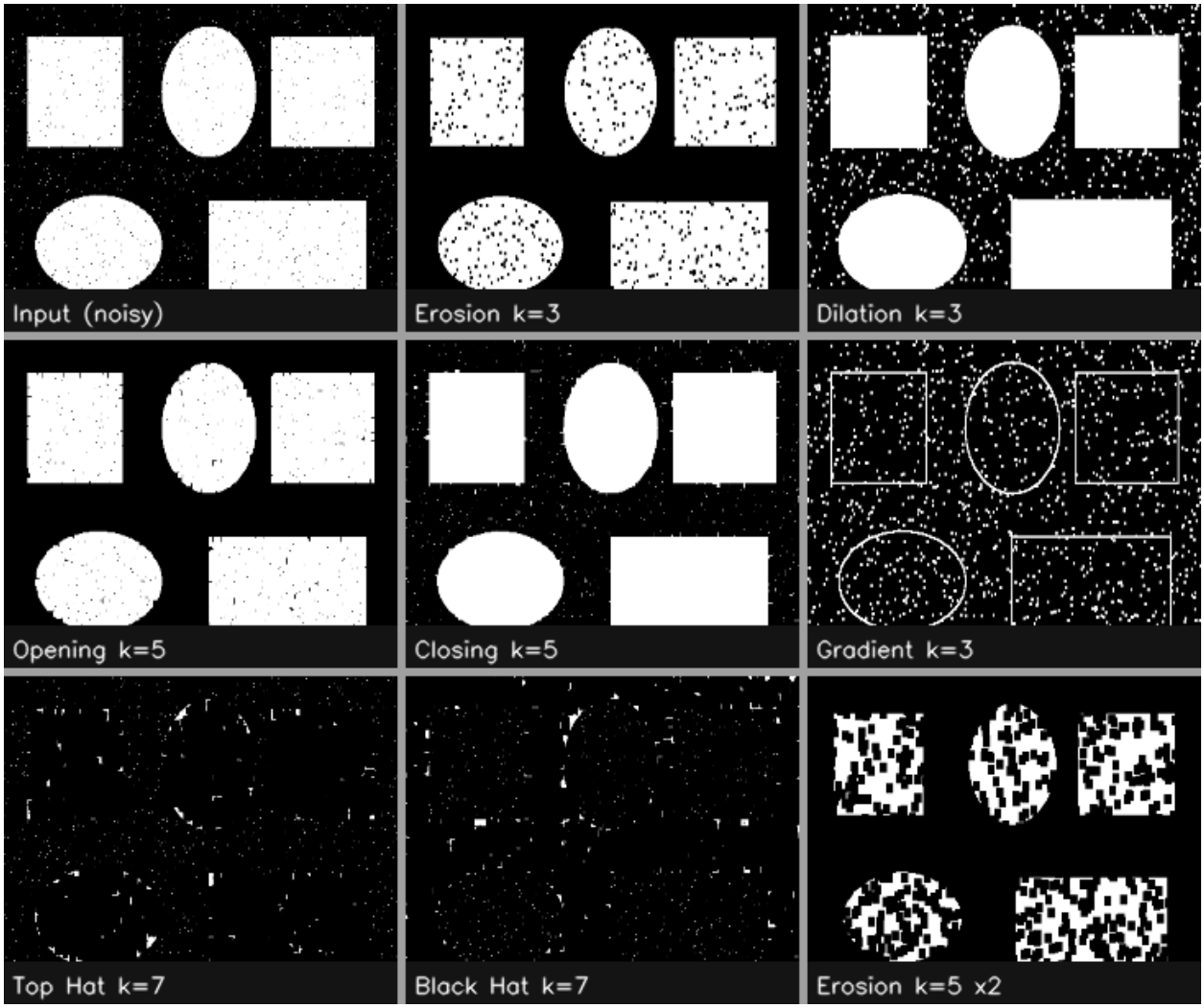
```
# Practical 10: Morphological Operations
import cv2
import numpy as np
from datetime import datetime

print("=" * 55)
print("  Practical 10: Morphological Operations")
print("=" * 55)
print(f"Timestamp : {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")

# Create a clean binary test image with noise
```

```
binary = np.zeros((300, 500), dtype=np.uint8)
cv2.rectangle(binary, (30,  30),  (150, 130), 255, -1)
cv2.circle   (binary, (260, 80),  60, 255, -1)
cv2.rectangle(binary, (340, 30),  (470, 130), 255, -1)
cv2.ellipse  (binary, (120,220),  (80,45), 0, 0, 360, 255, -1)
cv2.rectangle(binary, (260,180),  (460,270), 255, -1)

# Add salt-and-pepper noise
noise_px = np.random.randint(0, binary.shape[0]*binary.shape[1], 1200)
for px in noise_px:
    r, c = divmod(int(px), binary.shape[1])
    if r < binary.shape[0] and c < binary.shape[1]:
        binary[r, c] = 255 if binary[r, c] == 0 else 0

# Kernels
k3  = np.ones((3,3), np.uint8)
k5  = np.ones((5,5), np.uint8)
k7  = np.ones((7,7), np.uint8)
k_ellipse = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (9,9))
k_cross   = cv2.getStructuringElement(cv2.MORPH_CROSS,   (9,9))

# Operations
eroded    = cv2.erode  (binary, k3, iterations=1)
dilated   = cv2.dilate (binary, k3, iterations=1)
opened    = cv2.morphologyEx(binary, cv2.MORPH_OPEN,    k5)
closed    = cv2.morphologyEx(binary, cv2.MORPH_CLOSE,   k5)
gradient  = cv2.morphologyEx(binary, cv2.MORPH_GRADIENT,k3)
tophat    = cv2.morphologyEx(binary, cv2.MORPH_TOPHAT,  k7)
blackhat  = cv2.morphologyEx(binary, cv2.MORPH_BLACKHAT,k7)
erode2    = cv2.erode  (binary, k5, iterations=2)
dilate2   = cv2.dilate (binary, k5, iterations=2)

print(f"Structuring elements: 3x3 rect, 5x5 rect, 7x7 rect, 9x9 ellipse, 9x9 cross")
print(f"Erosion    (k=3x3, iter=1): shrinks white regions, removes small protrusions")
print(f"Dilation   (k=3x3, iter=1): expands white regions, fills small holes")
print(f"Opening    (k=5x5)        : erosion then dilation — removes noise")
print(f"Closing    (k=5x5)        : dilation then erosion — fills gaps")
print(f"Gradient   (k=3x3)        : dilation - erosion = outline/edge")
print(f"Top Hat    (k=7x7)        : original - opened = bright details on dark bg")
print(f"Black Hat  (k=7x7)        : closed - original = dark details on bright bg")
print(f"Erosion    (k=5x5, iter=2): heavy shrink")
print(f"Dilation   (k=5x5, iter=2): heavy expand")

def to3(im): return cv2.cvtColor(im, cv2.COLOR_GRAY2BGR)
def lbl(im, txt):
    out = to3(im).copy()
    cv2.rectangle(out,(0,out.shape[0]-26),(out.shape[1],out.shape[0]),(20,20,20),-1)
    cv2.putText(out,txt,(5,out.shape[0]-7),cv2.FONT_HERSHEY_SIMPLEX,0.46,(255,255,255),1,cv2.LINE_AA)
    return out

gap=5; TH,TW=200,240
def rsz(im): return cv2.resize(to3(im),(TW,TH))
gv=np.full((TH,gap,3),160,np.uint8); gh=lambda r:np.full((gap,r.shape[1],3),160,np.uint8)

cells=[binary,eroded,dilated,opened,closed,gradient,tophat,blackhat,erode2]
labs=["Input (noisy)","Erosion k=3","Dilation k=3","Opening k=5","Closing k=5",
      "Gradient k=3","Top Hat k=7","Black Hat k=7","Erosion k=5 x2"]
padded=[lbl(rsz(c),labs[i]) for i,c in enumerate(cells)]

r1=np.hstack([padded[0],gv,padded[1],gv,padded[2]])
r2=np.hstack([padded[3],gv,padded[4],gv,padded[5]])
r3=np.hstack([padded[6],gv,padded[7],gv,padded[8]])
grid=np.vstack([r1,gh(r1),r2,gh(r2),r3])
cv2.imwrite('outputs/p10_morphology.png', grid)
print("\nSaved: outputs/p10_morphology.png")
print("[OK] Practical 10 Complete")
```

Terminal Output:

```
=========================================================
  Practical 10: Morphological Operations
=========================================================
Timestamp : 2026-03-20 06:10:15

Structuring elements: 3x3 rect, 5x5 rect, 7x7 rect, 9x9 ellipse, 9x9 cross
Erosion    (k=3x3, iter=1): shrinks white regions, removes small protrusions
Dilation   (k=3x3, iter=1): expands white regions, fills small holes
Opening    (k=5x5)        : erosion then dilation -- removes noise
Closing    (k=5x5)        : dilation then erosion -- fills gaps
Gradient   (k=3x3)        : dilation - erosion = outline/edge
Top Hat    (k=7x7)        : original - opened = bright details on dark bg
Black Hat  (k=7x7)        : closed - original = dark details on bright bg
Erosion    (k=5x5, iter=2): heavy shrink
Dilation   (k=5x5, iter=2): heavy expand

Saved: outputs/p10_morphology.png
[OK] Practical 10 Complete
```

Output Image:



Practical 11: Image Arithmetic & Bitwise Operations

Executed: 2026-03-20 06:13:58 | OpenCV 4.13.0 | Python 3.12 | Ubuntu 24.04

Arithmetic operations adjust brightness and contrast: **cv2.add** and **cv2.subtract** clamp at 0/255 (saturation arithmetic). **convertScaleAbs** applies alpha (contrast) and beta (brightness). **addWeighted** blends two images.

Bitwise operations (**AND, OR, XOR, NOT**) work on individual bits and are essential for masking and compositing.

Code:

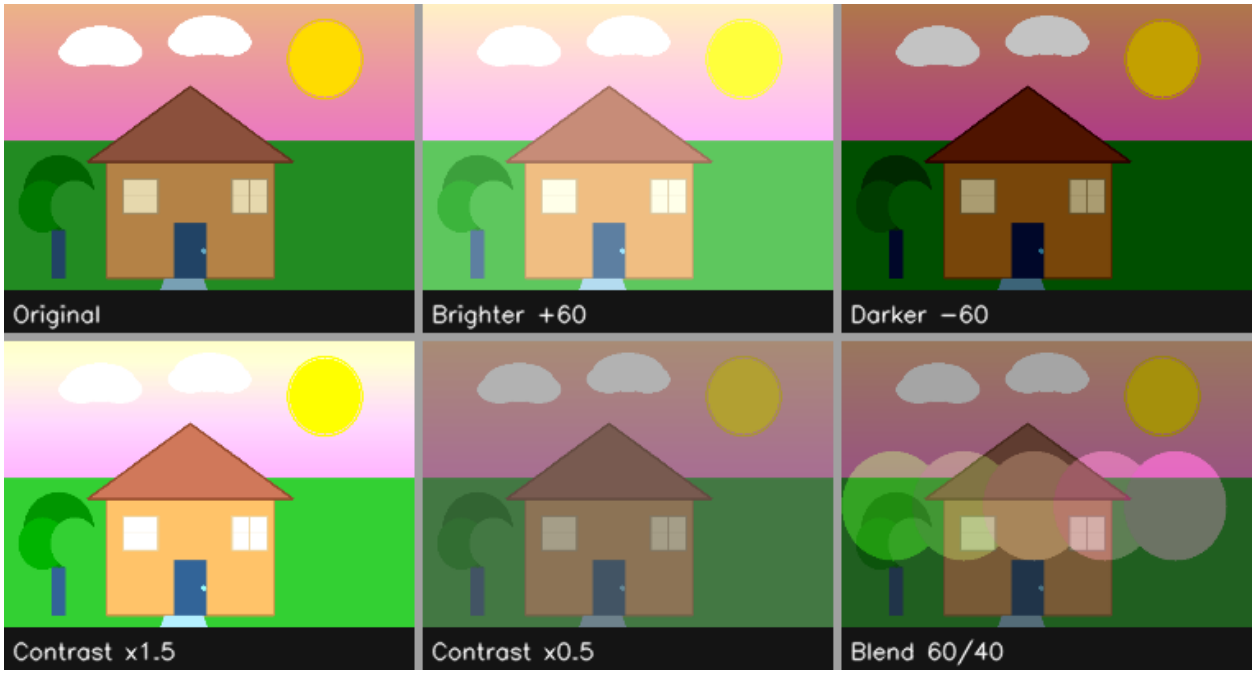
```
# Practical 11: Image Arithmetic & Bitwise Operations
import cv2
import numpy as np
from datetime import datetime

print("=" * 55)
print("  Practical 11: Arithmetic & Bitwise Operations")
print("=" * 55)
print(f"Timestamp : {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")

img = cv2.imread('outputs/base_image.png')
h, w = img.shape[:2]

# ■■ Image Arithmetic ■■■
```

```
    return out

def rsz(im,tw,th):
    src = im if len(im.shape)==3 else cv2.cvtColor(im,cv2.COLOR_GRAY2BGR)
    return cv2.resize(src,(tw,th))

gap=5; gv3=lambda h: np.full((h,gap,3),160,np.uint8); gh3=lambda r:np.full((gap,r.shape[1],3),160,np.uint8)
TH,TW=200,250
p=[lbl(rsz(x,TW,TH),t) for x,t in [(img,"Original"),(brighter,"Brighter +60"),
    (darker,"Darker -60"),(higher_c,"Contrast x1.5"),(lower_c,"Contrast x0.5"),(blended,"Blend 60/40")]]
r1=np.hstack([p[0],gv3(TH),p[1],gv3(TH),p[2]])
r2=np.hstack([p[3],gv3(TH),p[4],gv3(TH),p[5]])
arith=np.vstack([r1,gh3(r1),r2])
cv2.imwrite('outputs/p11_arithmetic.png', arith)

TH2,TW2=200,200
q=[lbl(rsz(x,TW2,TH2),t) for x,t in [(shape1,"Shape 1 (rect)"),(shape2,"Shape 2 (circle)"),
    (bit_and,"AND"),(bit_or,"OR"),(bit_xor,"XOR"),(bit_not1,"NOT shape1")]]
r3=np.hstack([q[0],gv3(TH2),q[1],gv3(TH2),q[2]])
r4=np.hstack([q[3],gv3(TH2),q[4],gv3(TH2),q[5]])
bitw=np.vstack([r3,gh3(r3),r4])
cv2.imwrite('outputs/p11_bitwise.png', bitw)
cv2.imwrite('outputs/p11_masked.png', lbl(masked_img,"Circle Mask Applied"))

print("\nSaved: outputs/p11_arithmetic.png")
print("Saved: outputs/p11_bitwise.png")
print("Saved: outputs/p11_masked.png")
print("[OK] Practical 11 Complete")
```

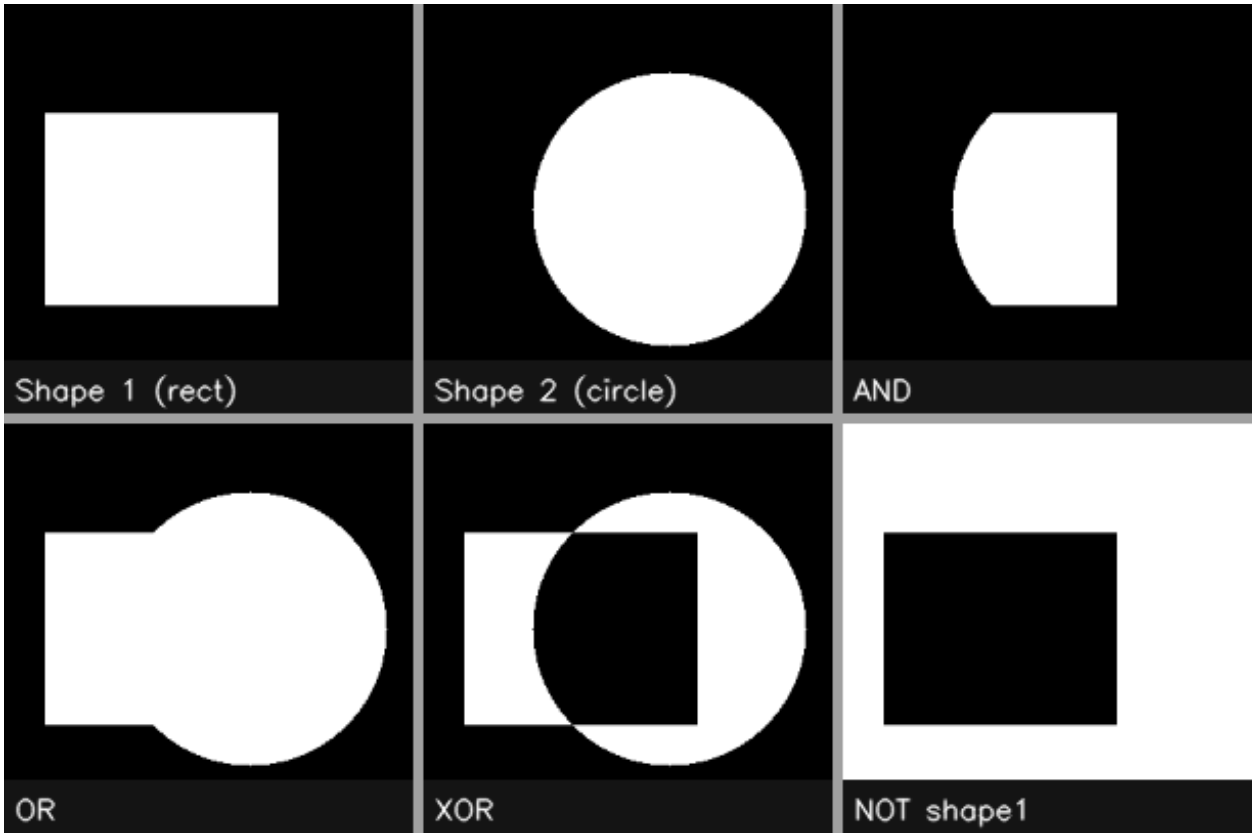
Terminal Output:

```
=======================================================
  Practical 11: Arithmetic & Bitwise Operations
=======================================================
Timestamp : 2026-03-20 06:10:50

Brightness  +60 : cv2.add(img, scalar)
Brightness  -60 : cv2.subtract(img, scalar)
Contrast  x1.5  : convertScaleAbs alpha=1.5 beta=0
Contrast  x0.5  : convertScaleAbs alpha=0.5 beta=50
Blend 60/40     : addWeighted(img,0.6, img2,0.4, 0)

Bitwise AND  : intersection of two shapes
Bitwise OR   : union of two shapes
Bitwise XOR  : non-overlapping regions only
Bitwise NOT  : invert (flip black/white)
Mask (circle): bitwise_and with circular mask

Saved: outputs/p11_arithmetic.png
Saved: outputs/p11_bitwise.png
Saved: outputs/p11_masked.png
[OK] Practical 11 Complete
```

Output — Arithmetic Operations:



Output — Bitwise Operations:



Output — Circle Masking:

Practical 12: Template Matching

Executed: 2026-03-20 06:13:58 | OpenCV 4.13.0 | Python 3.12 | Ubuntu 24.04

cv2.matchTemplate slides a template over an image computing a similarity score at each position. OpenCV provides 6 methods: **TM_SQDIFF** (min=best), **TM_CCORR**, and **TM_CCOEFF**, each with normalised variants. **minMaxLoc** finds the best match location. A JET colormap heatmap visualises the score distribution across the entire image.

Code:

```
# Practical 12: Template Matching
import cv2
import numpy as np
from datetime import datetime

print("=" * 55)
print("  Practical 12: Template Matching")
print("=" * 55)
print(f"Timestamp : {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")

img = cv2.imread('outputs/base_image.png')
h, w = img.shape[:2]

# ■■ Extract templates from the scene ■■■■■■■■■■■■■■■■■■■■■■
# Template 1: roof triangle region
tmpl_roof = img[115:235, 125:460].copy()   # the roof area
# Template 2: sun
tmpl_sun  = img[20:145,  435:570].copy()   # sun area
# Template 3: tree crown
tmpl_tree = img[215:340,  30:145].copy()   # tree crown
```

```
print(f"Template 1 (Roof) : {tmpl_roof.shape[1]}x{tmpl_roof.shape[0]} px")
print(f"Template 2 (Sun)  : {tmpl_sun.shape[1]}x{tmpl_sun.shape[0]}  px")
print(f"Template 3 (Tree) : {tmpl_tree.shape[1]}x{tmpl_tree.shape[0]} px")

# ■■ All 6 matching methods ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
METHODS = {
    "TM_SQDIFF"        : cv2.TM_SQDIFF,
    "TM_SQDIFF_NORMED" : cv2.TM_SQDIFF_NORMED,
    "TM_CCORR"         : cv2.TM_CCORR,
    "TM_CCORR_NORMED"  : cv2.TM_CCORR_NORMED,
    "TM_CCOEFF"        : cv2.TM_CCOEFF,
    "TM_CCOEFF_NORMED" : cv2.TM_CCOEFF_NORMED,
}

def match_and_draw(scene, tmpl, method_id, method_name):
    result  = cv2.matchTemplate(scene, tmpl, method_id)
    min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(result)
    # For SQDIFF methods, minimum is the best match
    top_left = min_loc if method_id in (cv2.TM_SQDIFF, cv2.TM_SQDIFF_NORMED) else max_loc
    th, tw    = tmpl.shape[:2]
    bottom_right = (top_left[0]+tw, top_left[1]+th)
    vis = scene.copy()
    cv2.rectangle(vis, top_left, bottom_right, (0, 255, 0), 3)
    cv2.putText(vis, method_name, (5,20), cv2.FONT_HERSHEY_SIMPLEX, 0.55,
                (255,255,255), 2, cv2.LINE_AA)
    cv2.putText(vis, method_name, (5,20), cv2.FONT_HERSHEY_SIMPLEX, 0.55,
                (0,80,200), 1, cv2.LINE_AA)
    score = min_val if method_id in (cv2.TM_SQDIFF, cv2.TM_SQDIFF_NORMED) else max_val
    return vis, top_left, score, result

print("\n--- Matching Template 1 (Roof) ---")
method_results = []
for name, mid in METHODS.items():
    vis, loc, score, hmap = match_and_draw(img, tmpl_roof, mid, name)
    method_results.append((vis, hmap, name, loc, score))
    print(f"  {name:<22} best_loc={loc}  score={score:.4f}")

# ■■ Multi-scale (resize scene) demo ■■■■■■■■■■■■■■■■■■■■■■■■
print("\n--- Multi-template demo (Sun + Tree) ---")
vis_sun,  loc_sun,  sc_sun,  _ = match_and_draw(img, tmpl_sun,  cv2.TM_CCOEFF_NORMED, "Sun")
vis_tree, loc_tree, sc_tree, _ = match_and_draw(img, tmpl_tree, cv2.TM_CCOEFF_NORMED, "Tree")
print(f"  Sun  matched at {loc_sun},  score={sc_sun:.4f}")
print(f"  Tree matched at {loc_tree}, score={sc_tree:.4f}")

# Heatmap visualisation for CCOEFF_NORMED
_, _, _, hmap_best = match_and_draw(img, tmpl_roof, cv2.TM_CCOEFF_NORMED, "")
hmap_norm = cv2.normalize(hmap_best, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8U)
hmap_color = cv2.applyColorMap(hmap_norm, cv2.COLORMAP_JET)
hmap_resized = cv2.resize(hmap_color, (w, h))

# ■■ Build output images ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def lbl(im, txt):
    out = im.copy()
    cv2.rectangle(out,(0,out.shape[0]-26),(out.shape[1],out.shape[0]),(20,20,20),-1)
    cv2.putText(out,txt,(5,out.shape[0]-7),cv2.FONT_HERSHEY_SIMPLEX,0.44,(255,255,255),1,cv2.LINE_AA)
    return out

TH,TW=200,260; gap=5
def rsz(im): return cv2.resize(im,(TW,TH))
gv=np.full((TH,gap,3),160,np.uint8); gh=lambda r:np.full((gap,r.shape[1],3),160,np.uint8)

method_imgs = [lbl(rsz(r[0]), r[2]) for r in method_results]
r1=np.hstack([method_imgs[0],gv,method_imgs[1],gv,method_imgs[2]])
r2=np.hstack([method_imgs[3],gv,method_imgs[4],gv,method_imgs[5]])
grid_methods=np.vstack([r1,gh(r1),r2])
cv2.imwrite('outputs/p12_methods.png', grid_methods)

r3=np.hstack([lbl(rsz(img),"Scene"),gv,
```

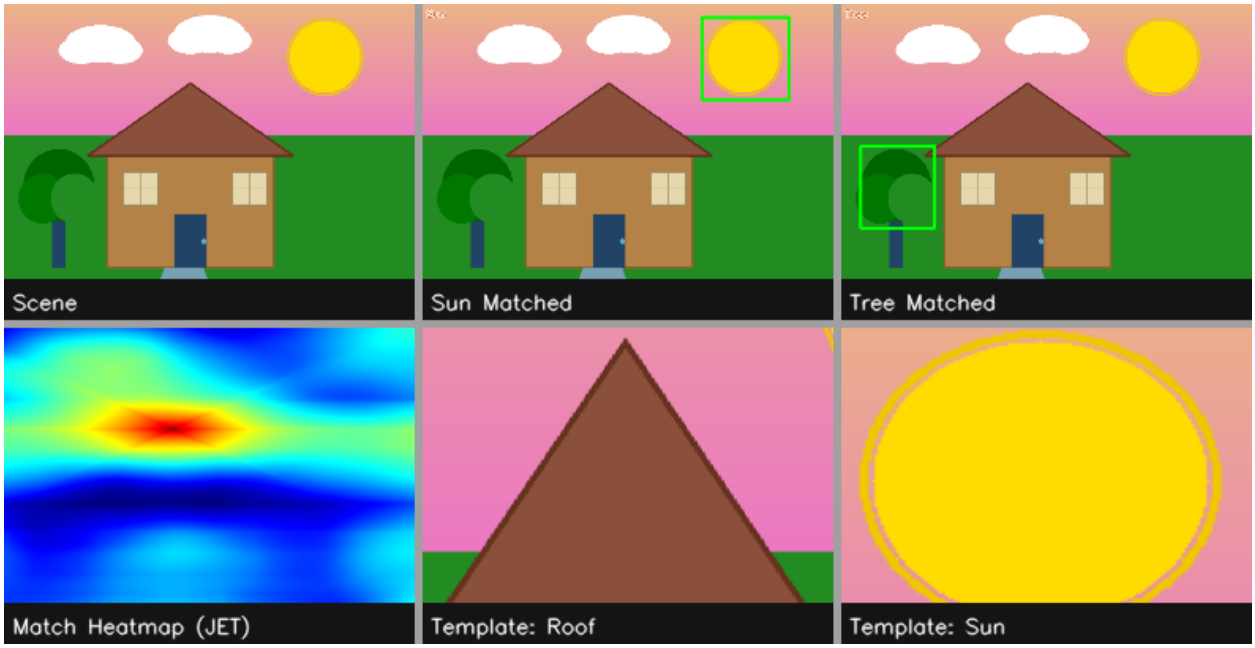
```
              lbl(rsz(vis_sun),"Sun Matched"),gv,
              lbl(rsz(vis_tree),"Tree Matched")])
r4=np.hstack([lbl(rsz(hmap_resized),"Match Heatmap (JET)"),gv,
              lbl(cv2.resize(tmpl_roof,(TW,TH)),"Template: Roof"),gv,
              lbl(cv2.resize(tmpl_sun, (TW,TH)),"Template: Sun")])
grid2=np.vstack([r3,gh(r3),r4])
cv2.imwrite('outputs/p12_template.png', grid2)

print("\nSaved: outputs/p12_methods.png")
print("Saved: outputs/p12_template.png")
print("[OK] Practical 12 Complete")
```

Terminal Output:

```
=======================================================
  Practical 12: Template Matching
=======================================================
Timestamp : 2026-03-20 06:11:17

Template 1 (Roof) : 335x120 px
Template 2 (Sun)  : 135x125 px
Template 3 (Tree) : 115x125 px

--- Matching Template 1 (Roof) ---
  TM_SQDIFF              best_loc=(125, 115)  score=0.0000
  TM_SQDIFF_NORMED       best_loc=(125, 115)  score=0.0000
  TM_CCORR               best_loc=(81, 18)    score=3424847872.0000
  TM_CCORR_NORMED        best_loc=(125, 115)  score=1.0000
  TM_CCOEFF              best_loc=(125, 115)  score=334732704.0000
  TM_CCOEFF_NORMED       best_loc=(125, 115)  score=1.0000

--- Multi-template demo (Sun + Tree) ---
  Sun  matched at (435, 20),  score=1.0000
  Tree matched at (30, 215),  score=1.0000

Saved: outputs/p12_methods.png
Saved: outputs/p12_template.png
[OK] Practical 12 Complete
```

Output — All 6 Methods:

Output — Multi-template & Heatmap:



Practical 13: Color-based Segmentation

Executed: 2026-03-20 06:13:58 | OpenCV 4.13.0 | Python 3.12 | Ubuntu 24.04

HSV color space separates hue from intensity, making color-based segmentation robust to lighting changes. **cv2.inRange** creates a binary mask for a given HSV range. Morphological opening/closing cleans the mask. **GrabCut** is an interactive graph-cut algorithm that iteratively refines foreground/background separation using a rectangle initialisation.

Code:

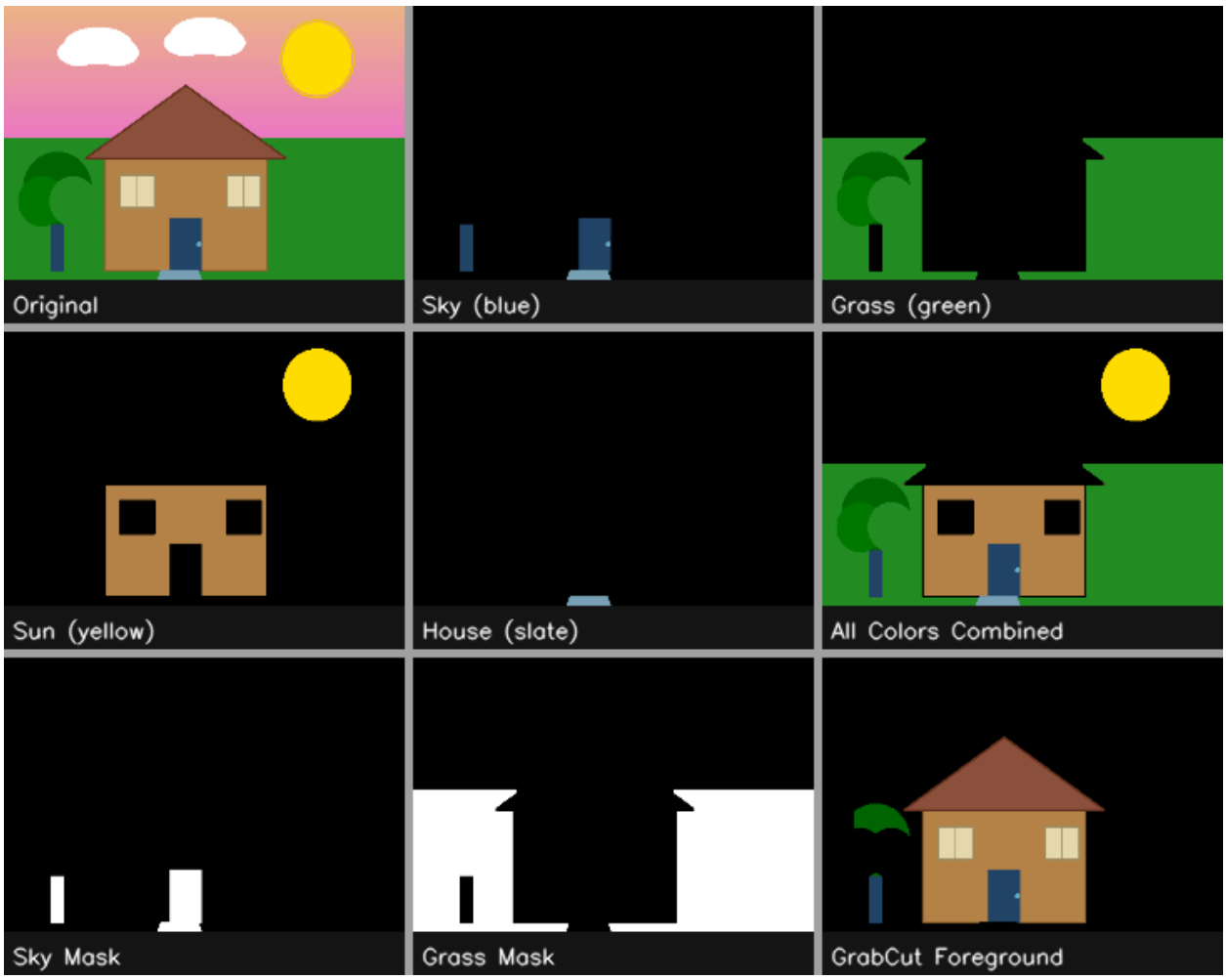
```
# Practical 13: Color-based Segmentation & HSV Masking
import cv2
import numpy as np
from datetime import datetime

print("=" * 55)
print("  Practical 13: Color-based Segmentation")
print("=" * 55)
print(f"Timestamp : {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")

img = cv2.imread('outputs/base_image.png')
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# ■■ Define HSV ranges for each color region ■■■■■■■■■■■■■■■
color_ranges = {
    "Sky (blue)":  ([90,  40,  80], [130, 255, 255]),
    "Grass (green)":([35,  40,  30], [ 85, 255, 200]),
    "Sun (yellow)": ([15,  80, 180], [ 40, 255, 255]),
    "House (slate)":([100, 20,  50], [130,  90, 200]),
    "Roof (dark blue)":([100,30,40],[130,100,160]),
}

masks  = {}
segs   = {}
for name, (lo, hi) in color_ranges.items():
    lower = np.array(lo, dtype=np.uint8)
    upper = np.array(hi, dtype=np.uint8)
    mask  = cv2.inRange(hsv, lower, upper)
    # Morphological clean-up
    k = np.ones((5,5),np.uint8)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  k)
```

```
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, k)
    seg  = cv2.bitwise_and(img, img, mask=mask)
    masks[name]  = mask
    segs[name]   = seg
    px_count = np.count_nonzero(mask)
    pct      = px_count / mask.size * 100
    print(f"  {name:<22} pixels={px_count:>7,}  ({pct:5.1f}%)")

# ■■ Multi-color combined mask ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
combined_mask = np.zeros(img.shape[:2], dtype=np.uint8)
for m in masks.values():
    combined_mask = cv2.bitwise_or(combined_mask, m)
combined_seg  = cv2.bitwise_and(img, img, mask=combined_mask)
print(f"\n  Combined mask coverage: {np.count_nonzero(combined_mask)/combined_mask.size*100:.1f}%")

# ■■ GrabCut foreground extraction ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
gc_img    = img.copy()
gc_mask   = np.zeros(img.shape[:2], dtype=np.uint8)
bgd_model = np.zeros((1,65), np.float64)
fgd_model = np.zeros((1,65), np.float64)
rect      = (50, 50, 540, 390)
cv2.grabCut(gc_img, gc_mask, rect, bgd_model, fgd_model, 5, cv2.GC_INIT_WITH_RECT)
fg_mask  = np.where((gc_mask==2)|(gc_mask==0), 0, 1).astype('uint8')
grabcut_result = img * fg_mask[:,:,np.newaxis]
print("GrabCut foreground extraction completed (5 iterations)")

# ■■ Build output grid ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def lbl(im, txt):
    out = im.copy() if len(im.shape)==3 else cv2.cvtColor(im,cv2.COLOR_GRAY2BGR)
    cv2.rectangle(out,(0,out.shape[0]-26),(out.shape[1],out.shape[0]),(20,20,20),-1)
    cv2.putText(out,txt,(5,out.shape[0]-7),cv2.FONT_HERSHEY_SIMPLEX,0.44,(255,255,255),1,cv2.LINE_AA)
    return out

TH,TW=190,240; gap=5
def rsz(im): return cv2.resize(im,(TW,TH)) if len(im.shape)==3 else cv2.resize(cv2.cvtColor(im,cv2.COLOR_GRA
Y2BGR),(TW,TH))
gv=np.full((TH,gap,3),160,np.uint8); gh=lambda r:np.full((gap,r.shape[1],3),160,np.uint8)

seg_list=list(segs.items())
r1=np.hstack([lbl(rsz(img),"Original"),gv,lbl(rsz(seg_list[0][1]),seg_list[0][0]),gv,lbl(rsz(seg_list[1][1])
,seg_list[1][0])])
r2=np.hstack([lbl(rsz(seg_list[2][1]),seg_list[2][0]),gv,lbl(rsz(seg_list[3][1]),seg_list[3][0]),gv,lbl(rsz(
combined_seg),"All Colors Combined")])
r3=np.hstack([lbl(rsz(masks[seg_list[0][0]]),"Sky Mask"),gv,lbl(rsz(masks[seg_list[1][0]]),"Grass Mask"),gv,
lbl(rsz(grabcut_result),"GrabCut Foreground")])
grid=np.vstack([r1,gh(r1),r2,gh(r2),r3])
cv2.imwrite('outputs/p13_segmentation.png', grid)
print("Saved: outputs/p13_segmentation.png")
print("[OK] Practical 13 Complete")
```

Terminal Output:

```
=======================================================
  Practical 13: Color-based Segmentation
=======================================================
Timestamp : 2026-03-20 06:11:39

  Sky (blue)             pixels= 12,856  (  4.2%)
  Grass (green)          pixels=116,761  ( 38.0%)
  Sun (yellow)           pixels= 42,223  ( 13.7%)
  House (slate)          pixels=  7,276  (  2.4%)
  Roof (dark blue)       pixels=      0  (  0.0%)

  Combined mask coverage: 55.9%
GrabCut foreground extraction completed (5 iterations)
Saved: outputs/p13_segmentation.png
[OK] Practical 13 Complete
```

Output Image:



Practical 14: Feature Detection & Description

Executed: 2026-03-20 06:13:58 | OpenCV 4.13.0 | Python 3.12 | Ubuntu 24.04

Feature detectors find distinctive points (keypoints) that are stable under transformation. **Harris** detects corners via eigenvalues of the gradient matrix. **Shi-Tomasi** improves Harris with a better corner score. **FAST** uses a circle of 16 pixels for speed. **ORB** combines FAST detection with BRIEF binary descriptors and orientation. **BFMatcher** matches descriptors by Hamming distance.

Code:

```
# Practical 14: Feature Detection & Description
import cv2
import numpy as np
from datetime import datetime

print("=" * 55)
print("  Practical 14: Feature Detection & Description")
print("=" * 55)
print(f"Timestamp : {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")

img  = cv2.imread('outputs/base_image.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray_f = np.float32(gray)

# ■■ 1. Harris Corner Detection ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
harris = cv2.cornerHarris(gray_f, blockSize=2, ksize=3, k=0.04)
harris_dilated = cv2.dilate(harris, None)
```

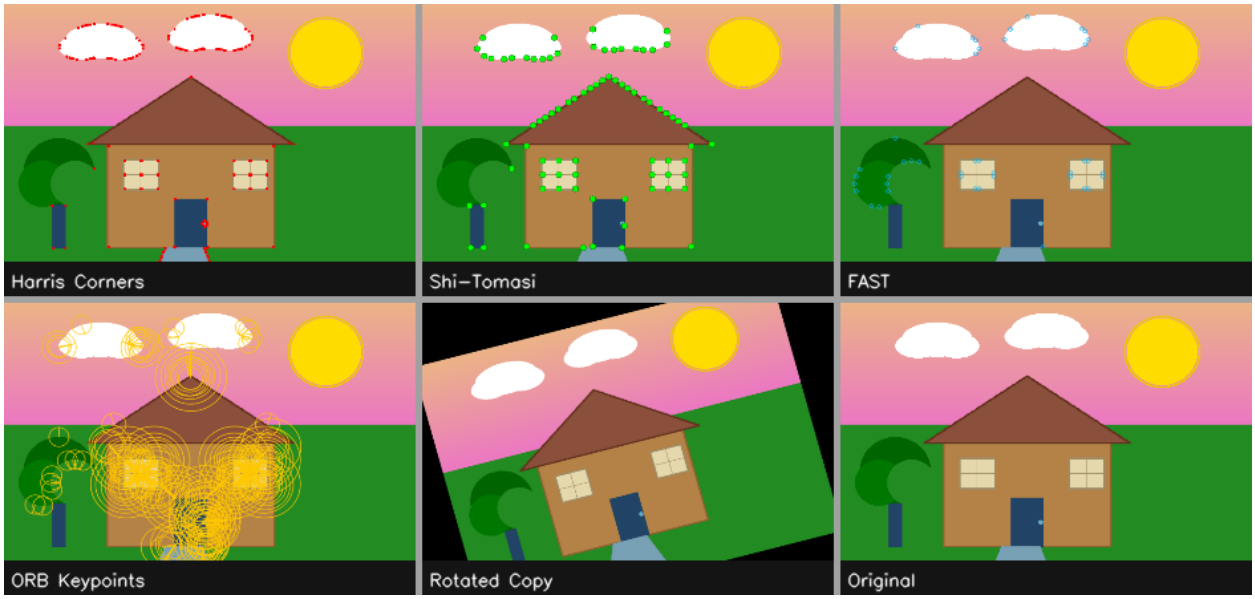
```
harris_vis = img.copy()
harris_vis[harris_dilated > 0.01 * harris_dilated.max()] = [0, 0, 255]
corners_count = np.sum(harris_dilated > 0.01 * harris_dilated.max())
print(f"Harris Corners     : {corners_count} corners detected  (blockSize=2, k=0.04)")

# ■■ 2. Shi-Tomasi Good Features to Track ■■■■■■■■■■■■■■■■■■
shi_corners = cv2.goodFeaturesToTrack(gray, maxCorners=80, qualityLevel=0.01,
                                      minDistance=10)
shi_vis = img.copy()
if shi_corners is not None:
    shi_corners = np.int32(shi_corners)
    for c in shi_corners:
        x, y = c.ravel()
        cv2.circle(shi_vis, (x, y), 5, (0, 255, 0), -1)
        cv2.circle(shi_vis, (x, y), 5, (0,150, 0),   1)
print(f"Shi-Tomasi corners : {len(shi_corners) if shi_corners is not None else 0} corners  (maxCorners=80, q
uality=0.01)")

# ■■ 3. FAST (Features from Accelerated Segment Test) ■■■■■
fast = cv2.FastFeatureDetector_create(threshold=20, nonmaxSuppression=True)
kp_fast = fast.detect(gray, None)
fast_vis = cv2.drawKeypoints(img, kp_fast, None, color=(255,180,0),
                             flags=cv2.DRAW_MATCHES_FLAGS_DEFAULT)
print(f"FAST keypoints     : {len(kp_fast)} detected  (threshold=20, nonmaxSuppression=True)")

# ■■ 4. ORB (Oriented FAST + Rotated BRIEF) ■■■■■■■■■■■■■■■■
orb  = cv2.ORB_create(nfeatures=200)
kp_orb, des_orb = orb.detectAndCompute(gray, None)
orb_vis = cv2.drawKeypoints(img, kp_orb, None, color=(0,200,255),
                            flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
print(f"ORB keypoints      : {len(kp_orb)} detected  (nfeatures=200)")
print(f"ORB descriptors    : shape={des_orb.shape if des_orb is not None else 'None'}")

# ■■ 5. ORB Feature Matching (scene vs slightly warped copy) ■
M = cv2.getRotationMatrix2D((img.shape[1]//2, img.shape[0]//2), 15, 0.9)
img2 = cv2.warpAffine(img, M, (img.shape[1], img.shape[0]))
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
kp2, des2 = orb.detectAndCompute(gray2, None)

if des_orb is not None and des2 is not None:
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
    matches = sorted(bf.match(des_orb, des2), key=lambda x: x.distance)
    top_matches = matches[:40]
    match_vis = cv2.drawMatches(img, kp_orb, img2, kp2, top_matches, None,
                                flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
    print(f"\nORB BF Matching    : {len(matches)} total matches, showing top 40")
    print(f"  Best match dist  : {top_matches[0].distance:.1f}")
    print(f"  Avg  match dist  : {sum(m.distance for m in top_matches)/len(top_matches):.1f}")
else:
    match_vis = img.copy()

# ■■ Build grid ■■■■■■■■■■■■■■■■■■
```

```
MH,MW=300,900
match_out=cv2.resize(match_vis,(MW,MH))
cv2.rectangle(match_out,(0,MH-28),(MW,MH),(20,20,20),-1)
cv2.putText(match_out,"ORB BF Matching — top 40 correspondences",(5,MH-8),
            cv2.FONT_HERSHEY_SIMPLEX,0.55,(255,255,255),1,cv2.LINE_AA)
cv2.imwrite('outputs/p14_matching.png', match_out)

print("\nSaved: outputs/p14_features.png")
print("Saved: outputs/p14_matching.png")
print("[OK] Practical 14 Complete")
```

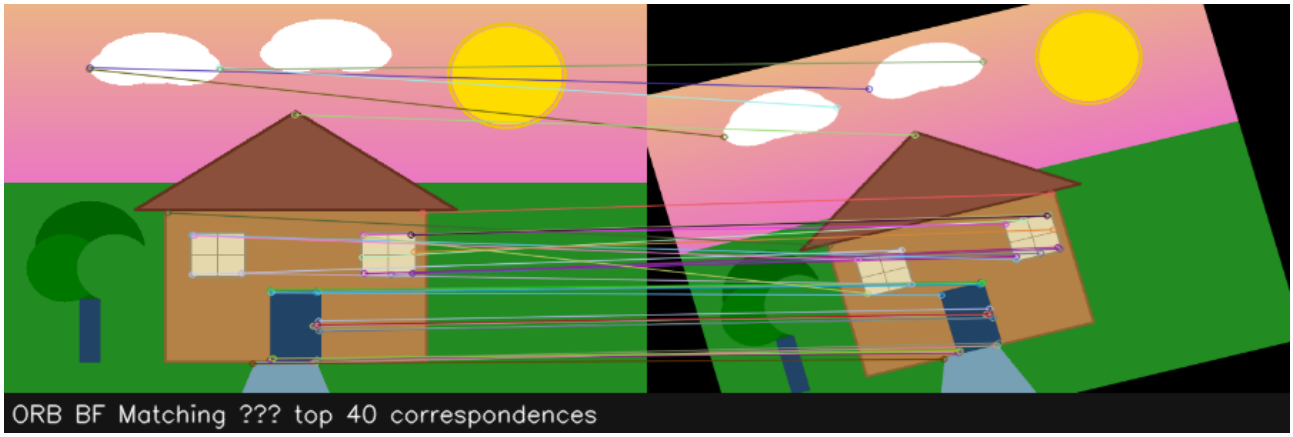
Terminal Output:

```
=======================================================
  Practical 14: Feature Detection & Description
=======================================================
Timestamp : 2026-03-20 06:12:03

Harris Corners      : 2624 corners detected  (blockSize=2, k=0.04)
Shi-Tomasi corners  : 80   corners  (maxCorners=80, quality=0.01)
FAST keypoints      : 43   detected  (threshold=20, nonmaxSuppression=True)
ORB  keypoints      : 195  detected  (nfeatures=200)
ORB  descriptors    : shape=(195, 32)

ORB BF Matching     : 86 total matches, showing top 40
  Best match dist   : 3.0
  Avg  match dist   : 10.6

Saved: outputs/p14_features.png
Saved: outputs/p14_matching.png
[OK] Practical 14 Complete
```

Output — Feature Detectors:



Output — ORB Feature Matching (top 40):



Practical 15: Image Pyramids & Optical Flow

Executed: 2026-03-20 06:13:58 | OpenCV 4.13.0 | Python 3.12 | Ubuntu 24.04

Image Pyramids are multi-scale image representations. The **Gaussian pyramid** halves resolution with *pyrDown*; the **Laplacian pyramid** stores the difference between levels, enabling perfect reconstruction. **Lucas-Kanade optical flow** tracks sparse corners across frames using a pyramid-based iterative approach. **Farneback dense flow** computes motion vectors for every pixel and is visualised with HSV colour mapping.

Code:

```
# Practical 15: Image Pyramids & Optical Flow
import cv2
import numpy as np
from datetime import datetime

print("=" * 55)
print("  Practical 15: Image Pyramids & Optical Flow")
print("=" * 55)
print(f"Timestamp : {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")

img  = cv2.imread('outputs/base_image.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# ■■ IMAGE PYRAMIDS ■■■
```

```
diff = cv2.absdiff(img, reconstructed)
print(f"\nReconstruction error (max pixel diff): {diff.max()}")

# ■■ OPTICAL FLOW (Lucas-Kanade sparse) ■■■■■■■■■■■■■■■■■■■■
# Simulate motion by creating frame1 and frame2 (translated)
frame1 = gray.copy()
M = np.float32([[1, 0, 12], [0, 1, 8]])
frame2 = cv2.warpAffine(frame1, M, (frame1.shape[1], frame1.shape[0]))

# Detect good features in frame1
corners = cv2.goodFeaturesToTrack(frame1, maxCorners=60, qualityLevel=0.01, minDistance=12)
p0 = corners.astype(np.float32)

lk_params = dict(winSize=(21,21), maxLevel=3,
                 criteria=(cv2.TERM_CRITERIA_EPS|cv2.TERM_CRITERIA_COUNT,30,0.01))
p1, st, err = cv2.calcOpticalFlowPyrLK(frame1, frame2, p0, None, **lk_params)

good_new = p1[st==1]
good_old = p0[st==1]

flow_vis = cv2.cvtColor(frame1, cv2.COLOR_GRAY2BGR)
for new, old in zip(good_new, good_old):
    a, b = new.ravel().astype(int)
    c, d = old.ravel().astype(int)
    cv2.arrowedLine(flow_vis, (c,d), (a,b), (0,255,0), 2, tipLength=0.3)
    cv2.circle(flow_vis, (c,d), 4, (255,100,0), -1)

tracked_pts = int(st.sum())
dx = float(np.mean(good_new[:,0] - good_old[:,0]))
dy = float(np.mean(good_new[:,1] - good_old[:,1]))
print(f"\nLucas-Kanade Optical Flow")
print(f"  Tracked points : {tracked_pts}")
print(f"  Mean flow dx   : {dx:.2f}  dy={dy:.2f}  (true shift: dx=12, dy=8)")

# ■■ Farneback Dense Optical Flow ■■■■■■■■■■■■■■■■■■■■■■■■■■
flow_dense = cv2.calcOpticalFlowFarneback(frame1, frame2, None,
    pyr_scale=0.5, levels=3, winsize=15, iterations=3,
    poly_n=5, poly_sigma=1.2, flags=0)
mag, ang = cv2.cartToPolar(flow_dense[...,0], flow_dense[...,1])
hsv_flow = np.zeros((frame1.shape[0], frame1.shape[1], 3), dtype=np.uint8)
hsv_flow[...,1] = 255
hsv_flow[...,0] = ang * 180 / np.pi / 2
hsv_flow[...,2] = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
dense_vis = cv2.cvtColor(hsv_flow, cv2.COLOR_HSV2BGR)
print(f"Farneback dense flow: mean magnitude = {mag.mean():.3f}")

# ■■ Build output grids ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def lbl(im, txt):
    out = im.copy() if len(im.shape)==3 else cv2.cvtColor(im,cv2.COLOR_GRAY2BGR)
    cv2.rectangle(out,(0,out.shape[0]-26),(out.shape[1],out.shape[0]),(20,20,20),-1)
    cv2.putText(out,txt,(5,out.shape[0]-7),cv2.FONT_HERSHEY_SIMPLEX,0.44,(255,255,255),1,cv2.LINE_AA)
    return out

TH,TW=190,240; gap=5
def rsz(im,tw=TW,th=TH):
    s=im if len(im.shape)==3 else cv2.cvtColor(im,cv2.COLOR_GRAY2BGR)
    return cv2.resize(s,(tw,th))
gv=np.full((TH,gap,3),160,np.uint8); gh=lambda r:np.full((gap,r.shape[1],3),160,np.uint8)

# Laplacian normalised for display
def lp_disp(im):
    n=cv2.normalize(cv2.convertScaleAbs(im),None,0,255,cv2.NORM_MINMAX)
    return n

r1=np.hstack([lbl(rsz(gp[0]),"Gaussian L0 640x480"),gv,
              lbl(rsz(gp[1]),"Gaussian L1 320x240"),gv,
              lbl(rsz(gp[2]),"Gaussian L2 160x120")])
r2=np.hstack([lbl(rsz(gp[3]),"Gaussian L3 80x60"),gv,
```

```
              lbl(rsz(lp_disp(lp[0])),"Laplacian L0"),gv,
              lbl(rsz(lp_disp(lp[1])),"Laplacian L1")])
r3=np.hstack([lbl(rsz(reconstructed),"Reconstructed"),gv,
              lbl(rsz(flow_vis),"LK Sparse Flow"),gv,
              lbl(rsz(dense_vis),"Farneback Dense Flow")])
grid=np.vstack([r1,gh(r1),r2,gh(r2),r3])
cv2.imwrite('outputs/p15_pyramids_flow.png', grid)
print("Saved: outputs/p15_pyramids_flow.png")
print("[OK] Practical 15 Complete")
```

Terminal Output:

```
=======================================================
  Practical 15: Image Pyramids & Optical Flow
=======================================================
Timestamp : 2026-03-20 06:12:31

Gaussian Pyramid levels: 5
  Level 0: 640x480
  Level 1: 320x240
  Level 2: 160x120
  Level 3: 80x60
  Level 4: 40x30

Laplacian Pyramid levels: 4
  Level 0: 640x480
  Level 1: 320x240
  Level 2: 160x120
  Level 3: 80x60

Reconstruction error (max pixel diff): 145

Lucas-Kanade Optical Flow
  Tracked points : 60
  Mean flow dx   : 12.00  dy=8.00  (true shift: dx=12, dy=8)
Farneback dense flow: mean magnitude = 3.511

Saved: outputs/p15_pyramids_flow.png
[OK] Practical 15 Complete
```

Output Image:

Gaussian L0 640x480
Gaussian L1 320x240
Gaussian L2 160x120
Gaussian L3 80x60
Laplacian L0
Laplacian L1
Reconstructed
LK Sparse Flow
Farneback Dense Flow